

Terraform

tumne ek baar terraform state file k baare mai bataya tha ki kaise hum state ko s3 mai rakhte hain aur dynamoDB mai lock karte hain par mujhe yaad nhi aa rha isko hum terraform se kaise karte hain, kya kya steps hote hain, aur agar 1 se jayada developers terraform code ko change kar rhe hain toh kaise state ko maintain kare, kyonki ho sakta hai dono developers ek saath state file mai change kare

Bilkul bhai. Confusion isliye ho raha hai kyonki yahan **do alag Terraform executions** hote hain: pehla remote-state S3 bucket banane ke liye, aur doosra actual AWS infrastructure banane ke liye. Main exact sequence scratch se samjhata hoon.

Complete Terraform state flow

Step 1: Terraform code likha

Step 2: terraform init

Step 3: terraform plan

Step 4: terraform apply

Step 5: Local terraform.tfstate bani

Step 6: S3 backend configure kiya

Step 7: Local state S3 mein migrate ki

Step 8: Future state directly S3 mein update hoti rahegi

Phase 1: Terraform se S3 state bucket banana

Sabse pehle humein ek S3 bucket chahiye jisme baad mein Terraform state rakhenge.

Lekin initial problem ye hai:

Terraform state store karne ke liye S3 bucket chahiye

S3 bucket abhi exist nahi karti

S3 bucket bhi Terraform se create karni hai

Isko **Terraform bootstrap process** kehte hain.

Step 1: Bootstrap folder banate hain

insurance-claim-devsecops/

└─ terraform/

└─ bootstrap/

│ └─ providers.tf

│ └─ main.tf

│ └─ outputs.tf

│

└─ infrastructure/

└─ backend.tf

└─ providers.tf

└─ main.tf

└─ variables.tf

Difference:

bootstrap/

└─ Sirf Terraform state bucket create karega

infrastructure/

└─ Actual application infrastructure create karega

Application infrastructure mein VPC, Lambda, API Gateway, DynamoDB, ECS, RDS jaise resources aa sakte hain.

Step 2: Bootstrap provider configuration

terraform/bootstrap/providers.tf:

```
terraform {  
  required_version = ">= 1.10.0"  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 6.0"  
    }  
  }  
}
```

```
provider "aws" {  
  region = "ap-south-1"  
}
```

Is file ka meaning:

- Terraform ka supported version define kiya
- AWS provider download karne ko bola
- AWS Mumbai region select kiya

Step 3: S3 state bucket ka Terraform code

terraform/bootstrap/main.tf:

```
resource "aws_s3_bucket" "terraform_state" {  
  bucket = "prashant-insurance-terraform-state"  
  lifecycle {  
    prevent_destroy = true  
  }  
  tags = {  
    Name = "Insurance Terraform State"  
    Environment = "Shared"  
    ManagedBy = "Terraform"  
  }  
}
```



```
resource "aws_s3_bucket_versioning" "terraform_state" {  
  bucket = aws_s3_bucket.terraform_state.id  
  versioning_configuration {
```

```

        status = "Enabled"
    }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "terraform_state" {
    bucket = aws_s3_bucket.terraform_state.id
    rule {
        apply_server_side_encryption_by_default {
            sse_algorithm = "AES256"
        }
    }
}

resource "aws_s3_bucket_public_access_block" "terraform_state" {
    bucket = aws_s3_bucket.terraform_state.id
    block_public_acls = true
    block_public_policy = true
    ignore_public_acls = true
    restrict_public_buckets = true
}

```

Is code se:

S3 bucket create hogi

Versioning enable hogi

Encryption enable hogi

Public access block hoga

Accidental Terraform destroy se bucket protected hogi

HashiCorp S3 state bucket par versioning enable karne ki strong recommendation deta hai, jisse accidental deletion ya human error ke case mein previous state recover ki ja sake. [\[developer....hicorp.com\]](https://developer.hashicorp.com/terraform/language/state/remote-state-s3)

Bucket name globally unique hona chahiye. Upar wala naam example hai, available na ho toh unique suffix lagana padega.

Step 4: Bootstrap Terraform initialize karna

Bootstrap folder mein jaate hain:

```
cd terraform/bootstrap
```

```
terraform init
```

terraform init kya karega?

AWS provider download karega

Current folder initialize karega

Backend configuration check karega

Abhi humne remote backend configure nahi kiya hai, isliye bootstrap configuration default **local backend** use karegi.

Is stage par state file abhi create nahi hoti. terraform init primarily working directory aur providers initialize karta hai.

Step 5: Terraform plan generate karna

```
terraform plan
```

Terraform batayega:

Plan: 4 to add, 0 to change, 0 to destroy

Ismein likely ye resources honge:

S3 bucket

S3 bucket versioning configuration

S3 encryption configuration

S3 public access block

terraform plan normally infrastructure create nahi karta. Ye proposed changes calculate karke dikhata hai.

Step 6: Terraform apply chalana

terraform apply

Approval ke liye Terraform puchega:

Do you want to perform these actions?

Enter a value:

Enter karo:

yes

Ab Terraform:

1. AWS APIs call karega
2. S3 bucket create karega
3. Versioning configure karega
4. Encryption configure karega
5. Public access block configure karega
6. Created resources ki details local state mein save karega

Step 7: Pehli local state file create hogi

Apply successful hone ke baad bootstrap folder mein ye file aayegi:

terraform/bootstrap/

└─ main.tf

└─ providers.tf

└─ terraform.tfstate

└─ terraform.tfstate.backup

Main state file:

terraform.tfstate

Ismein mapping hogi:

Terraform resource:

aws_s3_bucket.terraform_state

Actual AWS resource:

prashant-insurance-terraform-state

Yaani pehli local state **S3 bucket create karne wale bootstrap infrastructure ki state** hai.

Important:

S3 bucket Terraform se create ho gayi

Lekin bootstrap ki state abhi local computer/CloudShell mein hai

Phase 2: Actual infrastructure ki state S3 mein rakhna

Ab S3 state bucket AWS mein available hai. Ab hum actual infrastructure configuration ko bolenge:

Apni state local file mein nahi, is S3 bucket mein store karo.

Step 8: Main infrastructure mein backend file banana

terraform/infrastructure/backend.tf:

```
terraform {  
  backend "s3" {  
    bucket = "prashant-insurance-terraform-state"  
    key = "insurance-claim/dev/terraform.tfstate"  
    region = "ap-south-1"  
    encrypt = true  
    use_lockfile = true  
  }  
}
```

Property meanings:

bucket

└─ State kis S3 bucket mein store hogi

key

└─ Bucket ke andar state object ka path

region

└─ S3 bucket kis AWS region mein hai

encrypt

└─ State ko encrypted form mein store karna

use_lockfile

└─ Multiple Terraform operations ko same state par

simultaneously write karne se rokna

HashiCorp ke current S3 backend mein use_lockfile = true native S3 state locking enable karta hai. DynamoDB-based locking deprecated hai. [\[developer....hicorp.com\]](https://developer.hashicorp.com)

S3 ke andar object path hoga:

s3://prashant-insurance-terraform-state/

└─ insurance-claim/

└─ dev/

└─ terraform.tfstate

Locking ke waqt ek lock object bhi temporarily use ho sakta hai:

insurance-claim/dev/terraform.tfstate.tflock

Step 9: Actual infrastructure code likhna

For example, terraform/infrastructure/main.tf:

```
resource "aws_s3_bucket" "claim_documents" {  
    bucket = "prashant-insurance-claim-documents"  
    tags = {  
        Name = "Insurance Claim Documents"  
        Environment = "DEV"  
        ManagedBy = "Terraform"  
    }  
}
```

Ye actual application bucket hai.

Do buckets ko mix mat karna:

prashant-insurance-terraform-state

└─ Terraform state store karegi

prashant-insurance-claim-documents

└─ Insurance application documents store karegi

Step 10: Main infrastructure initialize karna

Ab infrastructure folder mein jaate hain:

```
cd ../infrastructure
```

```
terraform init
```

Terraform backend configuration read karega:

Backend type: S3

Bucket: prashant-insurance-terraform-state

Key: insurance-claim/dev/terraform.tfstate

Terraform successful message dega:

Successfully configured the backend "s3"!

Abhi state object S3 mein empty ya absent ho sakta hai, kyunki actual infrastructure ka apply abhi nahi hua.

Step 11: Main infrastructure plan

```
terraform plan
```

Terraform:

1. S3 remote backend check karega
2. Current remote state read karega
3. Terraform code read karega
4. AWS infrastructure check karega
5. Required changes ka plan banayega

Pehli baar output aa sakta hai:

Plan: 1 to add, 0 to change, 0 to destroy

Step 12: Main infrastructure apply

terraform apply

yes enter karne ke baad:

Terraform AWS resource create karega

↓

Resource ki ID aur details collect karega

↓

Remote state prepare karega

↓

State directly S3 backend mein save karega

Ab actual application state S3 mein hogi:

s3://prashant-insurance-terraform-state/

└─ insurance-claim/dev/terraform.tfstate

Is folder mein normal terraform.tfstate file create nahi hogi, kyunki backend remote S3 hai.

Pura process ek diagram mein

BOOTSTRAP PHASE

bootstrap/main.tf

↓

terraform init

↓

terraform plan

↓

terraform apply

↓

AWS mein state bucket create

↓

Bootstrap ki local terraform.tfstate create

MAIN INFRASTRUCTURE PHASE

infrastructure/backend.tf

↓

S3 remote backend configure

↓

terraform init

↓

terraform plan

↓

terraform apply

↓

AWS infrastructure create



Application state S3 mein store

Agar actual infrastructure pehle se local state ke saath bana hua ho

Suppose humne pehle infrastructure create kar diya:

infrastructure/

└─ terraform.tfstate

Aur baad mein S3 backend add kiya.

Tab normal terraform init ke bajaye migration command chalte hain:

terraform init -migrate-state

Terraform local state ko remote S3 backend mein copy/migrate karega:

Local terraform.tfstate



terraform init -migrate-state



S3 remote backend



insurance-claim/dev/terraform.tfstate

State migration ke baad verify karna chahiye:

terraform state list

Ye remote state mein managed resources dikhayega.

Multiple developers mein kya hota hai?

Assume Developer A aur Developer B same repository clone karte hain.

Dono ke backend.tf mein same configuration hai:

```
backend "s3" {  
  bucket = "prashant-insurance-terraform-state"  
  key = "insurance-claim/dev/terraform.tfstate"  
  region = "ap-south-1"  
  encrypt = true  
  use_lockfile = true  
}
```

Developer A apply chalata hai

Developer A



terraform apply



S3 lock acquire



Infrastructure update



S3 state update



Lock release

Isi waqt Developer B apply chalata hai

Developer B



terraform apply



Lock acquire karne ki attempt



Lock already Developer A ke paas



Developer B ka operation stop

Terraform supported backend par state-writing operations ke liye locking automatically perform karta hai. Lock acquire na hone par Terraform operation continue nahi karta. [\[developer....hicorp.com\]](#)

Developer A ka operation complete hone ke baad Developer B ko latest code pull karke dobara plan chalana chahiye:
git pull

terraform plan

Production best practice

Developers normally apni branches par code change karte hain:

Developer A branch └─

└─ Pull Request

Developer B branch └─



Validation and review



Main branch



CI/CD pipeline plan



Approval



CI/CD pipeline apply



S3 remote state and lock

Production mein individual developers ko directly terraform apply access dene ki jagah controlled CI/CD pipeline se apply karna safer hota hai.

DEV, QA aur PROD ka state separation

Same state file sab environments ke liye use nahi karenge.

S3 state bucket

└─ insurance-claim/dev/terraform.tfstate

└─ insurance-claim/qa/terraform.tfstate

└─ insurance-claim/prod/terraform.tfstate

DEV backend:

key = "insurance-claim/dev/terraform.tfstate"

QA backend:

key = "insurance-claim/qa/terraform.tfstate"

PROD backend:

key = "insurance-claim/prod/terraform.tfstate"

Isse DEV apply galti se PROD state modify nahi karega.

Sabse important distinction

Bootstrap state

└─ State S3 bucket ko track karti hai

Application state

└─ VPC, Lambda, ECS, RDS aur application resources track karti hai

Basic learning mein bootstrap state local reh sakti hai, lekin production-grade implementation mein bootstrap state ko bhi separately secure remote backend ya dedicated infrastructure-management process mein maintain kiya jata hai.

Short version yaad rakhne ke liye

1. Bootstrap Terraform local backend use karta hai
2. Bootstrap apply se S3 state bucket banti hai
3. Pehli bootstrap terraform.tfstate locally banti hai
4. Main project ke backend.tf mein S3 bucket configure hoti hai
5. Main project mein terraform init chalta hai
6. Main infrastructure ka apply hota hai
7. Main infrastructure ki state directly S3 mein store hoti hai
8. S3 lock concurrent apply ko prevent karta hai

jab hum apply chalte hain toh first time .tfstate file banti hai par second time kewal update hoti hai, agar 1 se jyada developer terraform par kaam kar rhe hain toh sabke system par .tfstate file banti hai, phir hum isko manage kaise karenge, kaise isko overwrite hone se bachayenge?

Bilkul, first apply par state create hoti hai aur subsequent applies par wahi state update hoti hai. **Lekin team environment mein har developer ko apni alag terraform.tfstate maintain nahi karni chahiye.** Sab developers ek hi centrally stored remote state use karte hain.

Pehle exact confusion clear karte hain

Local backend use karne par

Agar remote backend configure nahi hai, toh har developer ke system par alag state banegi:

Developer A

└─ main.tf

└─ terraform.tfstate

Developer B

└─ main.tf

└─ terraform.tfstate

Problem:

Developer A ki state → Version 5

Developer B ki state → Version 3

Dono states independent hongi. Developer A ko Developer B ke apply ki information nahi milegi.

Is scenario mein:

- Resources duplicate ho sakte hain
- Terraform existing resource ko naya samajh sakta hai
- Ek developer doosre developer ka resource remove kar sakta hai
- State aur AWS infrastructure inconsistent ho sakte hain

Isi wajah se team projects mein local state use nahi karte.

Remote S3 backend use karne par

Sab developers apne system par alag state create nahi karte. Sabka Terraform ek hi remote state access karta hai:

Developer A ─┐

Developer B ─┴─> S3 Remote State

Developer C ─┐

S3 mein ek centralized state object hota hai:

s3://company-terraform-state/insurance/dev/terraform.tfstate

Har developer ke backend.tf mein same backend configuration hoti hai:

```
terraform {  
  backend "s3" {  
    bucket = "company-terraform-state"  
    key = "insurance/dev/terraform.tfstate"  
    region = "ap-south-1"  
    encrypt = true  
    use_lockfile = true  
  }  
}
```

HashiCorp ke S3 backend mein state specified bucket aur key path par store hoti hai. Current Terraform versions mein use_lockfile = true S3-native state locking enable karta hai.

Kya developer ke system par .tfstate file banegi?

S3 backend successfully configure hone ke baad normally developer ke project folder mein application ki active terraform.tfstate file nahi banti.

Local system par primarily ye directory banegi:

.terraform/

Isme Terraform ki initialization aur backend-related metadata/cache hoti hai. Actual shared state S3 mein hoti hai:

Developer system

└─ main.tf

└─ variables.tf

└─ backend.tf

└─ .terraform/

└─ .terraform.lock.hcl

S3 bucket

└─ insurance/dev/terraform.tfstate

Yahan do “lock” ko mix mat karna:

.terraform.lock.hcl

Ye provider versions lock karti hai, jaise AWS provider ka version. Ye state locking nahi hai.

terraform.tfstate.tflck

Ye S3 native state lock ke liye use hoti hai jab:

use_lockfile = true

Multiple developers ka complete flow

Maan lo S3 state mein current infrastructure hai:

VPC

2 public subnets

2 private subnets

ALB

ECS cluster

Developer A change karta hai

Developer A ECS task count change karta hai:

desired_count = 3

Developer A command chalata hai:

terraform plan

terraform apply

Terraform ka flow:

1. S3 se latest state read

2. S3 state lock acquire

3. AWS infrastructure update

4. Updated state S3 mein write

5. State lock release

Usi samay Developer B apply karta hai

Developer B ALB configuration change kar raha hai:

terraform apply

Lekin Developer A ke paas state lock hai:

Developer A → Lock acquired

Developer B → Lock unavailable

Developer B ko error milega:

Error acquiring the state lock

Developer B ka Terraform state overwrite nahi karega. Terraform operation ko stop kar dega.

Terraform supported backend par state-writing operations ke liye locking automatically perform karta hai. Lock acquire nahi hone par operation continue nahi hota.

Developer A complete hone ke baad

Developer A ka apply successful hone par:

AWS infrastructure update

↓

S3 state update

↓

Lock release

Ab S3 state latest version par hai.

Developer B ko latest Git code lena chahiye:

git pull

terraform plan

Terraform dobara:

1. S3 se updated state read karega
2. AWS ki current condition refresh karega
3. Developer B ke code se compare karega
4. Naya plan dikhayega

Plan verify karne ke baad:

terraform apply

Ab Developer B lock acquire karke apna change apply kar sakta hai.

State overwrite hone se kaun bachata hai?

State ki protection multiple layers se hoti hai.

1. S3 remote backend

Sab developers ko ek common state deta hai:

One environment

↓

One central state

Isse har developer ki independent local state nahi hoti.

2. State locking

Ek time par sirf ek Terraform write operation allow hota hai:

Developer A apply → Allowed

Developer B apply → Blocked

Lock state ko simultaneous write aur corruption se bachata hai.

3. S3 bucket versioning

S3 versioning enable karni chahiye:

```
resource "aws_s3_bucket_versioning" "terraform_state" {  
  bucket = aws_s3_bucket.terraform_state.id  
  versioning_configuration {  
    status = "Enabled"  
  }  
}
```

Har state update par S3 previous version preserve kar sakta hai:

terraform.tfstate Version 1

terraform.tfstate Version 2

terraform.tfstate Version 3

Agar latest state accidentally corrupt ya delete ho jaye, previous S3 version recovery mein help kar sakta hai.

HashiCorp bhi accidental deletion aur human error se recovery ke liye S3 bucket versioning strongly recommend karta hai.

4. Git and pull requests

Terraform code ko developers directly main branch mein overwrite nahi karte:

Developer A branch —> Pull Request —┐

└─> Main branch

Developer B branch —> Pull Request —┐

Agar same Terraform file ki same lines modify hui hain, Git merge conflict show karega.

Git Terraform **code conflict** manage karta hai. S3 locking Terraform **state conflict** manage karti hai.

5. CI/CD pipeline

Production mein ideally developers directly apply nahi chalte.

Developer

↓

Feature branch

↓

Pull request

↓

terraform fmt

↓

terraform validate

↓

terraform plan

↓

Review and approval

↓

Main branch merge

↓

CI/CD terraform apply

↓

S3 state update

Ek controlled pipeline apply chalati hai. Isse multiple developers ke direct applies aur accidental production changes reduce hote hain.

Lock overwrite kaise rokta hai?

Without locking:

10:00 → Developer A reads State Version 10

10:00 → Developer B reads State Version 10

10:05 → Developer A writes State Version 11

10:06 → Developer B writes its result over Version 11

Developer B ki write Developer A ke update ko overwrite kar sakti hai.

Locking ke saath:

10:00 → Developer A acquires lock

10:01 → Developer A reads latest state

10:02 → Developer B requests lock

10:02 → Developer B is blocked

10:05 → Developer A writes updated state

10:05 → Developer A releases lock

10:06 → Developer B retries

10:06 → Developer B reads latest updated state

Isliye Developer B old state ke upar blindly write nahi karta.

Important: Lock code conflict solve nahi karta

Suppose Developer A ne code mein kiya:

```
instance_type = "t3.medium"
```

Developer B ne same resource par kiya:

```
instance_type = "t3.large"
```

State lock sirf ye ensure karega ki dono ka apply simultaneously na ho.

Lekin agar Developer A ke baad Developer B ka code apply hua, toh final instance type t3.large ho sakta hai. Isko technical state corruption nahi, balki **code/change coordination problem** kahenge.

Isliye locking ke saath ye bhi required hain:

- Feature branches
- Pull requests
- Code review
- Latest git pull
- Fresh terraform plan
- Controlled CI/CD apply
- Production approval

State file Git mein commit nahi karni

.gitignore mein ye entries honi chahiye:

.terraform/

.tfstate

.tfstate.

crash.log

crash..log

*.tfplan

Lekin normally ye file commit honi chahiye:

.terraform.lock.hcl

Reason:

terraform.tfstate

→ Infrastructure state hai

→ Sensitive values bhi ho sakti hain

→ Git mein commit nahi karni

.terraform.lock.hcl

→ Provider versions lock karti hai

→ Normally Git mein commit karni hai

Agar pehle sab developers ke paas local state hai toh?

Agar remote backend se pehle local state use ho rahi thi, toh developers ko randomly apni-apni state S3 mein upload nahi karni chahiye.

Pehle identify karo:

Kaunsi state actual/complete AWS infrastructure ko correctly represent karti hai?

Us verified state ko migrate karo:

terraform init -migrate-state

Uske baad verify karo:

terraform state list

terraform plan

Expected result ideally:

No changes. Your infrastructure matches the configuration.

Phir baaki developers:

git pull

terraform init -reconfigure

terraform plan

Ab sab S3 ki same remote state use karenge. Purani local state files ko Git mein commit ya manually combine nahi karna chahiye.

DEV, QA aur PROD ke liye separate states

Multiple environments ko same state object mein mix nahi karna chahiye:

S3 bucket

└─ insurance/dev/terraform.tfstate

└─ insurance/qa/terraform.tfstate

└─ insurance/prod/terraform.tfstate

Meaning:

DEV developers/pipeline → DEV state

QA pipeline → QA state

PROD pipeline → PROD state

Ye environment isolation provide karta hai.

Interview-ready answer

In a team environment, I do not maintain separate Terraform state files on every developer's machine. I configure an S3 remote backend so that all developers and CI/CD pipelines use the same centralized state. I enable S3 native state locking using `use_lockfile = true`, which allows only one state-writing operation at a time.

When one developer runs Terraform apply, Terraform acquires the lock. If another developer attempts an apply at the same time, Terraform cannot acquire the lock and stops the second operation. After the first apply completes, Terraform updates the state in S3 and releases the lock.

I also enable S3 versioning for state recovery, keep separate state keys for DEV, QA, and PROD, and use pull requests and a controlled CI/CD pipeline so production applies are serialized and reviewed.

Abhi ka actionable point

Bas ye line yaad rakho:

Multiple developers ki multiple state files nahi hoti.

Multiple developers ek centralized S3 remote state use karte hain.

S3 lock concurrent overwrite se bachata hai.

Git/PR code conflict se bachata hai.

CI/CD pipeline production apply control karti hai.

terraform init working directory initialize karta hai aur providers/modules download karta hai.

terraform fmt formatting standardize karta hai.

terraform validate configuration syntax aur internal consistency check karta hai.

terraform plan proposed infrastructure changes show karta hai.

terraform apply approved changes execute karta hai.

terraform destroy Terraform-managed resources remove karta hai.

Resource aur data source mein difference kya hai?

Resource block infrastructure object create ya manage karta hai, jabki data source existing information read karta hai.

For example, `aws_vpc` resource ek VPC create kar sakta hai. `data "aws_vpc"` existing VPC ki details retrieve karta hai.

Example:

```
resource "aws_s3_bucket" "claims" {  
  bucket = "insurance-claims-dev"  
}  
  
data "aws_caller_identity" "current" {}
```

First apply aur second apply mein state ke saath kya hota hai?

First apply par Terraform resources create karta hai aur initial state generate karta hai. Subsequent apply par Terraform existing state read karta hai, actual infrastructure refresh karta hai, configuration ke saath compare karta hai aur same state ko latest infrastructure information ke saath update karta hai.

Team environment mein main state ko local systems par maintain nahi karta. Main centralized remote backend use karta hoon.

Terraform operations se pehle state ko real infrastructure ke against refresh karta hai aur required changes identify karta hai.

Remote backend kyon use karte hain?

Remote backend ka use centralized state storage ke liye hota hai. Agar multiple developers Terraform par kaam kar rahe hain, toh sab ek common remote state access karte hain.

AWS environment mein main S3 remote backend use karta hoon, S3 versioning enable karta hoon, encryption configure karta hoon aur state locking enable karta hoon.

Remote backend collaboration improve karta hai aur state ko individual developer systems par dependent hone se bachata hai. Non-local backend use hone par Terraform normally state ko local disk par persist nahi karta, except remote write failure jaise recovery cases.

Terraform state locking kya hai?

State locking concurrent state writes ko prevent karta hai. Jab ek developer ya pipeline terraform apply chala raha hota hai, Terraform state lock acquire karta hai.

Agar second operation same state ko modify karne ki koshish kare, toh lock acquire nahi hota aur second operation stop ho jata hai. Isse state corruption aur simultaneous overwrite prevent hota hai.

Terraform supported backend par write operations ke liye locking automatically perform karta hai. Lock acquire na hone par operation continue nahi hota.

State file ko Git mein commit karna chahiye?

Nahi. Main terraform.tfstate ko Git mein commit nahi karta. State mein resource IDs, infrastructure metadata aur potentially sensitive information store ho sakti hai.

Main state ko access-controlled remote backend mein store karta hoon aur Terraform state files ko .gitignore mein add karta hoon.

```
.terraform/  
*.tfstate  
*.tfstate.*  
*.tfplan  
crash.log
```

Stale state lock ko kaise remove karte hain?

Pehle main verify karta hoon ki koi Terraform apply actually run nahi ho raha. Agar previous operation crash hone ke baad stale lock reh gaya hai, tab lock ID ke saath terraform force-unlock use karta hoon.

```
terraform force-unlock LOCK_ID
```

Main active operation ka lock forcefully remove nahi karta, kyunki isse multiple writers aur state corruption ho sakti hai.

HashiCorp force-unlock ko carefully aur sirf failed unlocking scenario mein use karne ki warning deta hai.

terraform apply -lock=false use kar sakte hain?

Technically option available ho sakta hai, lekin collaborative ya production environment mein main -lock=false use nahi karta. Lock disable karne se concurrent writes state ko corrupt kar sakti hain.

Agar lock issue aa raha hai, main pehle running pipeline, stale lock aur backend connectivity investigate karta hoon.

HashiCorp locking disable karne ko recommend nahi karta.

State corrupt ho jaye toh kya karoge?

Main pehle new applies stop karunga aur remote state ka backup lunga. S3 versioning enabled ho toh previous known-good state version inspect ya recover karunga.

Main actual AWS resources aur Terraform configuration compare karunga. Zarurat hone par terraform import, terraform state rm, terraform state mv ya recovery workflow use karunga. State file ko directly edit karna avoid karunga.

Input variables, locals aur outputs mein difference kya hai?

Input variables external values receive karti hain, jaise environment name ya instance type. Locals configuration ke andar repeated expressions ko simplify karte hain. Outputs created resource values expose karte hain, jaise ALB DNS name ya VPC ID.

```
variable "environment" {
```

```
  type = string
```

```
}
```

```
locals {
```

```
  name_prefix = "insurance-${var.environment}"
```

```
}
```

```
output "bucket_name" {
```

```
  value = aws_s3_bucket.claims.id
```

```
}
```

Terraform module kya hai?

Terraform module reusable Terraform configuration ka collection hota hai. Main repeated infrastructure patterns, jaise VPC, ECS service, ALB aur RDS, ko reusable modules mein organize karta hoon.

Root module child modules call karta hai aur environment-specific values variables ke through pass karta hai.

Example structure:

```
terraform/
```

```
├── modules/
```

```
|   ├── vpc/
```

```
|   ├── alb/
```

```
|   └── ecs-service/
```

```
└── environments/
```

```
    ├── dev/
```

```
    ├── qa/
```

```
    └── prod/
```

Count aur for_each mein kya difference hai?

count numeric index based multiple resource instances create karta hai. for_each map ya set ke stable keys ke basis par resources create karta hai.

Main identical fixed-number resources ke liye count use kar sakta hoon, lekin individually identifiable resources ke liye for_each prefer karta hoon.

count example:

```
resource "aws_subnet" "private" {  
  count = 2  
  vpc_id = aws_vpc.main.id  
  cidr_block = "10.10.${count.index + 1}.0/24"  
}
```

for_each example:

```
resource "aws_subnet" "private" {  
  for_each = {  
    app = "10.10.1.0/24"  
    db = "10.10.2.0/24"  
  }  
}
```

```
vpc_id = aws_vpc.main.id  
cidr_block = each.value  
}
```

depends_on kab use karte ho?

Terraform resource references ke basis par implicit dependency graph automatically banata hai. Agar dependency attribute reference se visible nahi hai, tab main depends_on se explicit dependency define karta hoon.

Main unnecessary depends_on avoid karta hoon, kyunki ye dependency graph ko conservative bana sakta hai.

Resource ko destroy kiye bina Terraform management se kaise remove karoge?

```
terraform state rm aws_s3_bucket.example
```

Interview answer:

terraform state rm resource ki state binding remove karta hai, actual cloud resource ko delete nahi karta. Is command se pehle main state backup leta hoon aur downstream dependencies check karta hoon.

State se remove hone ke baad agar resource block configuration mein present raha, toh next plan usse dobara create karne ka proposal de sakta hai.

Existing AWS resource ko Terraform mein kaise manage karoge?

Main pehle matching Terraform resource configuration likhunga. Uske baad resource ko Terraform state mein import karunga aur terraform plan chala kar configuration ko actual resource ke saath align karunga.

CLI example:

```
terraform import aws_s3_bucket.claims existing-bucket-name
```

Modern configuration-driven approach:

```
import {  
  to = aws_s3_bucket.claims  
  id = "existing-bucket-name"  
}
```

DEV, QA aur PROD state kaise separate karoge?

Main har environment ke liye separate state key maintain karta hoon.

insurance/dev/terraform.tfstate
insurance/qa/terraform.tfstate
insurance/prod/terraform.tfstate

Production-grade isolation:

DEV account → DEV pipeline → DEV state

QA account → QA pipeline → QA state

PROD account → PROD pipeline → PROD state

Isse blast radius reduce hota hai aur DEV apply PROD infrastructure ko modify nahi karta.

Terraform secrets kaise handle karte ho?

Main secrets ko .tf files, tfvars files ya Git repository mein hardcode nahi karta. Main AWS Secrets Manager, SSM Parameter Store, environment variables ya CI/CD secret storage use karta hoon.

Sensitive variables ko sensitive = true mark karta hoon, lekin mujhe pata hai ki ye value ko CLI output mein hide karta hai, state se necessarily remove nahi karta. Isliye remote state encryption aur access control zaruri hai.

Example:

```
variable "db_password" {  
  type = string  
  sensitive = true  
}
```

Terraform ko AWS credentials kaise doge?

Main access keys Terraform files mein hardcode nahi karta. Local approved environment mein AWS profile use ho sakti hai. CI/CD mein main OIDC federation aur AWS STS temporary role credentials prefer karta hoon.

Pipeline least-privilege IAM role assume karti hai aur short-lived credentials use karti hai.

Backend configuration mein credentials directly include karna recommended nahi hai. Conventional credential files, environment variables ya workload identity methods prefer kiye jaate hain.

Terraform state bucket ko kaise secure karoge?

Main S3 public access block, encryption, versioning, least-privilege bucket policy, CloudTrail auditing aur state locking configure karta hoon.

State bucket ko application bucket se separate rakhta hoon. Production state access sirf authorized CI/CD role aur limited administrative roles ko deta hoon.

Terraform CI/CD pipeline ka flow kya hota hai?

Feature branch aur pull request par main formatting, validation, security scanning aur Terraform plan chalata hoon. Plan reviewers ko infrastructure impact dikhata hai.

Approved merge ke baad controlled pipeline apply chalati hai. Production apply ke liye manual approval aur protected environment use karta hoon.

Typical flow:

Feature branch

↓

Pull request

↓
terraform fmt -check
↓
terraform validate
↓
TFLint / Checkov
↓
terraform plan
↓
Review and approval
↓
Main branch merge
↓
terraform apply

Plan-on-PR aur apply-after-approval pattern state consistency aur change governance improve karta hai. CI/CD integration, remote state aur recovery scenarios senior Terraform interviews mein key focus areas hain.

Multiple developers Terraform code par kaise work karte hain?

Developers separate Git feature branches par Terraform code change karte hain. Pull request par plan generate hota hai aur code review hota hai.

Terraform code conflicts ko Git manage karta hai. State write conflicts ko remote backend locking manage karti hai.

Production apply controlled CI/CD pipeline perform karti hai.

Git/PR

→ Code conflict manage karta hai

Remote state locking

→ Concurrent state write manage karti hai

CI/CD approval

→ Deployment governance manage karta hai

terraform apply halfway fail ho jaye toh kya hota hai?

Terraform transactional database ki tarah complete rollback guarantee nahi karta. Jo resources successfully create ho chuke hain, woh exist kar sakte hain aur successful state updates record ho sakte hain.

Main error identify karta hoon, AWS aur state status verify karta hoon, code fix karta hoon aur fresh terraform plan run karta hoon. Terraform remaining difference calculate karega.

Strong response:

Main blindly state delete ya complete destroy nahi karta. Pehle partial infrastructure and state reconciliation karta hoon.

AWS Console se resource delete ho gaya toh kya hoga?

Next refresh ya plan mein Terraform identify karega ki state mein resource managed hai, lekin actual AWS resource missing hai. Agar configuration mein resource abhi bhi defined hai, Terraform normally usse recreate karne ka plan dega.

Apply se pehle main verify karta hoon ki deletion accidental thi ya intentional.

AWS Console se configuration change ho gayi toh kya karoge?

Terraform plan drift show karega. Agar manual change unauthorized ya accidental hai, main Terraform apply se declared configuration restore karta hoon.

Agar change valid hai, pehle Terraform code update karke PR review karata hoon, phir approved plan apply karta hoon.

Terraform dependency graph kya hai?

Terraform resource references ke basis par dependency graph banata hai. Independent resources parallel create ho sakte hain aur dependent resources correct order mein create hote hain.

Main direct references prefer karta hoon, jaise subnet mein `aws_vpc.main.id`, kyunki isse implicit dependency automatically establish ho jaati hai.

Provisioners kyon avoid karne chahiye?

Provisioners last-resort option hone chahiye. Terraform infrastructure lifecycle model karta hai, lekin arbitrary scripts ke behaviour aur failure ko reliably model nahi kar pata.

Main software configuration ke liye baked images, cloud-init, AWS Systems Manager, user data, configuration-management tools ya container images prefer karta hoon.

Terraform purpose-built solutions ko provisioners par prefer karta hai, kyunki provisioners direct network access, credentials, unpredictable behaviour aur security complexity introduce kar sakte hain.

Multi-region deployment Terraform se kaise manage karoge?

Main same AWS provider ke multiple aliased configurations define karta hoon aur respective resources ya modules ko correct provider pass karta hoon.

```
provider "aws" {  
  region = "ap-south-1"  
}
```

```
provider "aws" {  
  alias = "secondary"  
  region = "ap-southeast-1"  
}
```

```
resource "aws_s3_bucket" "secondary" {  
  provider = aws.secondary  
  bucket = "insurance-secondary-region"  
}
```

Multiple provider configurations multi-region resources manage karne ka common pattern hain. Child module ko alternate provider providers mapping ke through pass kiya ja sakta hai.

Implicit and explicit dependency :

Dependency ka matlab hai: **kaunsa AWS resource pehle create hoga aur kaunsa uske baad**. Terraform resource references aur depends_on ke basis par dependency graph banata hai, jisse creation, update aur destruction ka correct order decide hota hai.

Terraform dependency kya hoti hai?

Maan lo humein create karna hai:

VPC

↓

Subnet

↓

EC2 instance

Correct sequence:

1. Pehle VPC create hogi
2. Phir VPC ke andar subnet create hoga
3. Phir subnet ke andar EC2 instance create hoga

Terraform ko ye order dependency se pata chalta hai.

Terraform dependencies do types ki hoti hain:

1. Implicit dependency
2. Explicit dependency

1. Implicit dependency kya hoti hai?

Implicit dependency automatically Terraform detect karta hai jab ek resource, doosre resource ki ID, ARN, name ya kisi output ko directly reference karta hai.

Example:

```
resource "aws_vpc" "main" {  
  cidr_block = "10.0.0.0/16"  
  tags = {  
    Name = "insurance-vpc"  
  }  
}
```

```
resource "aws_subnet" "private" {  
  vpc_id = aws_vpc.main.id  
  cidr_block = "10.0.1.0/24"  
  tags = {  
    Name = "insurance-private-subnet"  
  }  
}
```

Subnet resource mein ye reference dekho: vpc_id = aws_vpc.main.id

Iska matlab:

```
aws_subnet.private
```

↓ depends on

```
aws_vpc.main
```

Terraform automatically samajh jayega:

Subnet ko create karne ke liye VPC ID required hai, isliye pehle VPC create karni padegi.

Humein manually depends_on likhne ki zarurat nahi hai. Terraform configuration mein resource attribute reference ko parse karke automatically dependency graph mein edge create karta hai.

Creation order

```
aws_vpc.main
```

↓

```
aws_subnet.private
```

Destruction order reverse hoga

```
aws_subnet.private
```

↓

```
aws_vpc.main
```

Terraform pehle subnet destroy karega, phir VPC destroy karega, kyunki VPC ke andar existing subnet hone par VPC delete nahi ki ja sakti.

2. Explicit dependency kya hoti hai?

Kabhi-kabhi do resources ke beech dependency hoti hai, lekin code mein direct ID, ARN ya attribute reference nahi hota.

Is situation mein Terraform dependency automatically identify nahi kar pata.

Tab hum explicitly likhte hain:

```
depends_on = [  
  resource_name  
]
```

depends_on ek Terraform meta-argument hai jo hidden dependency define karta hai. Iska use tab hota hai jab ek resource doosre resource ke behaviour par depend karta ho, lekin uska data directly arguments mein reference nahi karta.

Explicit dependency ka simple example

Maan lo pehle S3 bucket ready honi chahiye, uske baad Lambda function create karna hai. Lekin Lambda resource S3 bucket ki ID ya ARN directly use nahi kar raha.

10-second memory trick

Implicit = Terraform khud samajhta hai

because one resource references another

Explicit = Hum Terraform ko batate hain

using depends_on

Is example ko yaad karo:

Implicit dependency

```
subnet_id = aws_subnet.private.id
# Explicit dependency
depends_on = [aws_iam_role_policy.s3_access]
```

How does Terraform determine resource creation order?

Terraform builds a dependency graph from resource attribute references and explicit `depends_on` declarations. It creates dependent resources in the correct order and can create independent resources in parallel. During destroy, Terraform generally follows the reverse dependency order.

Terraform resource-reference interpolations ko dependencies mein convert karta hai aur `depends_on` declarations ko graph edges ke roop mein include karta hai.

Interview-ready short answer

Terraform supports implicit and explicit dependencies.

An implicit dependency is automatically created when one resource references an attribute of another resource. For example, when a subnet uses `aws_vpc.main.id`, Terraform understands that the VPC must be created before the subnet.

An explicit dependency is defined using the `depends_on` meta-argument. I use it only when there is a hidden behavioural dependency that Terraform cannot infer from resource references. For example, an ECS task may need an IAM policy to be fully attached before the task starts, even though the task definition does not directly reference the policy.

I prefer implicit dependencies because they produce a more accurate dependency graph and allow Terraform to process independent resources in parallel. I use explicit dependencies only when necessary.

Bilkul bhai. `for_each` ka purpose hai: **ek hi Terraform resource block se multiple resources banana, lekin har resource ko ek meaningful unique key/name se identify karna.** Terraform `for_each` ko maps aur sets ke saath use kar sakta hai, aur har created resource ko alag manage karta hai.

Sabse pehle simple example

Maan lo tumhe AWS mein teen S3 buckets banani hain:

claims bucket

policies bucket

documents bucket

Bina `for_each` ke tum teen resource blocks likhoge:

```
resource "aws_s3_bucket" "claims" {
```

```
  bucket = "insurance-claims-dev"
```

```
}
```

```
resource "aws_s3_bucket" "policies" {
```

```
  bucket = "insurance-policies-dev"
```

```
}
```

```
resource "aws_s3_bucket" "documents" {
```

```
  bucket = "insurance-documents-dev"
```

```
}
```

Code repeat ho raha hai.

`for_each` ke saath ek hi block se teen buckets bana sakte ho:

```
resource "aws_s3_bucket" "insurance_buckets" {  
  for_each = toset([  
    "insurance-claims-dev",  
    "insurance-policies-dev",  
    "insurance-documents-dev"  
  ])  
  bucket = each.value  
}
```

Terraform is block ko teen baar logically execute karega:

First iteration:

each.value = insurance-claims-dev

Second iteration:

each.value = insurance-policies-dev

Third iteration:

each.value = insurance-documents-dev

Result:

insurance-claims-dev

insurance-policies-dev

insurance-documents-dev

each.key aur each.value kya hain?

for_each use karne par Terraform do special values provide karta hai:

each.key

each.value

Inhe loop ke current item ki key aur value samjho.

Map ke saath for_each

Map ka format hota hai:

```
{  
  key1 = "value1"  
  key2 = "value2"  
}
```

Example:

```
locals {  
  bucket_names = {  
    claims = "insurance-claims-dev"  
    policies = "insurance-policies-dev"  
    documents = "insurance-documents-dev"  
  }  
}
```

Yahan:

claims → key

insurance-claims-dev → value

policies → key

insurance-policies-dev → value

Resource:

```
resource "aws_s3_bucket" "insurance_buckets" {
```

```
  for_each = local.bucket_names
```

```
  bucket = each.value
```

```
  tags = {
```

```
    Name = each.key
```

```
    Project = "Insurance"
```

```
  }
```

```
}
```

```
}
```

Terraform har iteration mein values aise read karega:

First iteration

each.key = claims

each.value = insurance-claims-dev

Second iteration

each.key = policies

each.value = insurance-policies-dev

Third iteration

each.key = documents

each.value = insurance-documents-dev

Result:

Bucket: insurance-claims-dev

Tag: Name = claims

Bucket: insurance-policies-dev

Tag: Name = policies

Bucket: insurance-documents-dev

Tag: Name = documents

Insurance project ka practical example

Maan lo tumhe Claim Service aur Policy Service ke liye separate security groups banane hain.

```
locals {
```

```
  microservices = {
```

```
    claim-service = 8000
```

```
    policy-service = 8001
```

```
  }}
```

Resource:

```
resource "aws_security_group" "microservice_sg" {  
  for_each = local.microservices  
  name = "${each.key}-sg"  
  vpc_id = aws_vpc.main.id  
  ingress {  
    from_port = each.value  
    to_port = each.value  
    protocol = "tcp"  
    cidr_blocks = ["10.0.0.0/16"]  
  }  
  tags = {  
    Name = "${each.key}-sg"  
    Service = each.key  
  }  
}
```

Terraform do security groups create karega:

claim-service-sg

Port: 8000

policy-service-sg

Port: 8001

First iteration

each.key = claim-service

each.value = 8000

Terraform effective configuration:

name = "claim-service-sg"

from_port = 8000

to_port = 8000

Second iteration

each.key = policy-service

each.value = 8001

Terraform effective configuration:

name = "policy-service-sg"

from_port = 8001

to_port = 8001

Terraform state mein resources kaise dikhenge?

for_each se resources numeric positions ki jagah keys se identify hote hain:

aws_security_group.microservice_sg["claim-service"]

aws_security_group.microservice_sg["policy-service"]

State list command:

terraform state list

Output roughly:

```
aws_security_group.microservice_sg["claim-service"]
```

```
aws_security_group.microservice_sg["policy-service"]
```

Yahi for_each ka bada benefit hai. Resource ko meaningful key se identify kar sakte ho.

Kisi ek resource ko kaise reference karenge?

Claim Service security group:

```
aws_security_group.microservice_sg["claim-service"].id
```

Policy Service security group:

```
aws_security_group.microservice_sg["policy-service"].id
```

ECS service example:

```
resource "aws_ecs_service" "claim_service" {  
  name = "claim-service"  
  network_configuration {  
    security_groups = [  
      aws_security_group.microservice_sg["claim-service"].id  
    ]  
  }  
}
```

Terraform implicit dependency bhi samajh jayega:

Claim Service security group

↓

Claim ECS Service

Kyunki ECS service security group ki ID ko directly reference kar rahi hai.

Set aur map mein difference

Set example

```
for_each = toset([  
  "claims",  
  "policies",  
  "documents"  
])
```

Set ke case mein generally:

each.key = claims

each.value = claims

Key aur value same hoti hain.

Resource:

```
resource "aws_s3_bucket" "example" {  
  for_each = toset([  
    "claims",  
    "policies"  
  ])
```

```
bucket = "insurance-#{each.value}-dev"
```

```
}
```

Result:

```
insurance-claims-dev
```

```
insurance-policies-dev
```

Map example

```
for_each = {
```

```
  claim = 8000
```

```
  policy = 8001
```

```
}
```

Map ke case mein:

```
each.key = claim
```

```
each.value = 8000
```

Map tab useful hota hai jab har item ke saath separate configuration value ho.

Complex map ka practical example

Maan lo har microservice ke liye sirf port nahi, balki CPU aur memory bhi define karni hai:

```
locals {
```

```
  microservices = {
```

```
    claim = {
```

```
      port = 8000
```

```
      cpu = 256
```

```
      memory = 512
```

```
    }
```

```
    policy = {
```

```
      port = 8001
```

```
      cpu = 512
```

```
      memory = 1024
```

```
    }
```

```
  }
```

```
}
```

ECS task definition:

```
resource "aws_ecs_task_definition" "microservice" {
```

```
  for_each = local.microservices
```

```
  family = "${each.key}-service"
```

```
  cpu = each.value.cpu
```

```
  memory = each.value.memory
```

```
  network_mode = "awsvpc"
```

```
  requires_compatibilities = ["FARGATE"]
```

```
  container_definitions = jsonencode([
```

```
    {
```

```
      name = "${each.key}-service"
```

```
      image = "example-image"
```

```
essential = true
portMappings = [
{
containerPort = each.value.port
protocol = "tcp"
}}]}}
```

Claim iteration:

```
each.key = claim
each.value.port = 8000
each.value.cpu = 256
each.value.memory = 512
```

Policy iteration:

```
each.key = policy
each.value.port = 8001
each.value.cpu = 512
each.value.memory = 1024
```

Result:

claim-service

CPU: 256

Memory: 512

Port: 8000

policy-service

CPU: 512

Memory: 1024

Port: 8001

count aur for_each mein difference

count

```
resource "aws_instance" "server" {
count = 3
instance_type = "t3.micro"
}
```

Resources numeric index se identify honge:

aws_instance.server[0]

aws_instance.server[1]

aws_instance.server[2]

count fixed number of similar resources ke liye easy hai. Terraform count.index ko zero-based numeric index deta hai.

for_each

```
resource "aws_instance" "server" {
for_each = {
claim = "t3.micro"
policy = "t3.small"
}
```

```
}  
instance_type = each.value  
tags = {  
  Name = each.key  
}
```

Resources keys se identify honge:

```
aws_instance.server["claim"]
```

```
aws_instance.server["policy"]
```

Easy difference

count

→ Resources ko number se identify karta hai

→ server[0], server[1]

for_each

→ Resources ko meaningful key se identify karta hai

→ server["claim"], server["policy"]

for_each ko kab prefer karna chahiye?

for_each tab use karo jab:

- Har resource ka meaningful unique name ho
- Resources ki configurations different hon
- Ek specific item ko add ya remove karna ho
- Resource ko stable key se identify karna ho
- Map ke through port, CPU, memory ya CIDR pass karna ho

Example:

claim-service

policy-service

notification-service

Yahan for_each better hai because names meaningful hain.

Resource remove karne par kya hota hai?

Suppose current map:

```
microservices = {  
  claim = 8000  
  policy = 8001  
}
```

State:

```
microservice_sg["claim"]
```

```
microservice_sg["policy"]
```

Ab tum policy remove karte ho:

```
microservices = {  
  claim = 8000  
}
```

Next plan mein Terraform specifically ye identify karega:

microservice_sg["policy"] will be destroyed

Claim resource unaffected rehega:

microservice_sg["claim"] remains unchanged

Yahi stable keys ka advantage hai.

Important caution: key rename karna

Suppose pehle key thi:

```
claim = 8000
```

Tumne key rename kar di:

```
claim-service = 8000
```

Terraform normally ise aise interpret kar sakta hai:

Old resource:

```
microservice_sg["claim"]
```

→ Destroy

New resource:

```
microservice_sg["claim-service"]
```

→ Create

Terraform ke liye resource address change ho gaya.

Agar actual resource ko replace nahi karna aur sirf logical key rename karni hai, toh moved block use kiya ja sakta hai:

```
moved {
```

```
  from = aws_security_group.microservice_sg["claim"]
```

```
  to = aws_security_group.microservice_sg["claim-service"]
```

```
}
```

Interview-ready answer

for_each is a Terraform meta-argument that I use to create multiple resource or module instances from a map or set.

Each instance is identified by a unique key.

Inside the resource block, each.key represents the current item's key and each.value represents its value. For example, I can define Claim Service and Policy Service in a map with their respective ports and create separate security groups from a single resource block.

I prefer for_each when resources have meaningful and stable identities, such as claim-service and policy-service. Their Terraform addresses become resource.name["claim-service"] and resource.name["policy-service"]. In comparison, count identifies resources using numeric indexes such as [0] and [1].

Bas ye example yaad rakho

```
locals {
```

```
  services = {
```

```
    claim = 8000
```

```
policy = 8001
```

```
}}
```

```
resource "aws_security_group" "service" {
```

```
  for_each = local.services
```

```
  name = "${each.key}-sg"
```

```
  ingress {
```

```
    from_port = each.value
```

```
    to_port = each.value
```

```
    protocol = "tcp"
```

```
  }}
```

Memory trick:

for_each = list/map ke har item ke liye resource banao

each.key = current resource ka naam

each.value = current resource ki configuration

Abhi ka actionable step: Upar wale example mein sirf itna identify karo: claim aur policy **keys** hain, jabki 8000 aur 8001 unki **values** hain. यही for_each ka core concept hai.